9044 python学习

基础语法

1 sys

```
"""
1. sys.argv
sys.argv是一个列表，例如: sys.argv = ['echo.py', 'hello', 'world']
sys.argv[0] 是脚本的名称，sys.argv[1] 是第一个参数
"""
len(sys.argv)  # 获取参数个数 $#
sys.argv  # 获取所有参数 $@

"""
2.
sys.stdin

sys.stdin.read() -> 返回一个字符串 "hello\nworld\npython\n"

sys.stdin.readlines() -> 返回列表  ['hello\n', 'world\n', 'python\n']

echo hello | python3 add.py 1 3 也就是管道进入程序的会进入sys.stdin
-> sys.argv = ['add.py', '1', '3']
-> print(sys.stdin.read()) = 'hello\n'
"""

"""
3. sys.exit(1) 系统错误退出
"""


```

2 try...except...

基础语法

```
```

```
try:
  可能发生异常的代码
except 异常状态 as e # 如果发生错误，就把错误存在变量e中
  print(e)
  处理异常的代码块
else:
  没有发生异常时执行的代码块
```

常见异常状态与案例

```
"""ValueError"""
try:
    x = int("abc")
except ValueError:
    print("发生了 ValueError：无法把 abc 转成整数")

"""FileNotFoundError"""
try:
    f = open("hello.txt", "r")
except FileNotFoundError:
    print("发生了 FileNotFoundError：文件不存在")

"""KeyError"""
try:
    d = {"name": "Tom"}
    print(d["age"])
except KeyError:
    print("发生了 KeyError：字典里没有这个键")

"""IndexError"""
try:
    lst = [1, 2, 3]
    print(lst[5])
except IndexError:
    print("发生了 IndexError：列表索引越界")

"""TyprError"""
try:
    print(3 + "5")
except TypeError:
    print("发生了 TypeError：类型错误")
```

文件读写有关的异常状态

1. 文件不存在

2. 没有权限读
3. 路径写错
4. 系统问题

```
try:
    file = open("data", "r")
    content = file.read()
    file.close()
    print(content)
except FileNotFoundError:
    print("文件不存在")
except PermissionError:
    print("没有权限读取文件")
except OSError as e:
    print("其他系统错误: ", e)
```

3 文件读写操作

基本文件操作 open close

```
# 读模式
file = open("a.txt", "r")
data = file.read()

# 写模式
file = open("a.txt", "w")
file.write(data)
# 追加
file = open("a.txt", "a")
file.write(data)

# 追加二进制
file = open("a.txt", "ab")

# 关闭文件
file.close()
```

with open

```python
# 不用写f.close()
with open("a.txt", "r") as f:
    data = f.read()
    print(data)
```

4 字典

```python
# 通过key检索
d = {"name": "Tom", "age": 18}
print(d["name"])   # Tom

# 删除键值对
d = {"a": 1, "b": 2}
del d["a"]
print(d)   # {"b": 2}

# 清空字典
d.clear()

# 输出所有键
d = {"a": 1, "b": 2}
print(d.keys())

# 输出所有键值对
d = {"a": 1, "b": 2}
print(d.items())

# 输出所有值
d = {"a": 1, "b": 2}
print(d.values())
```

5 subprocess.run()

通过这个函数，调用外部命令，例如：ls sort

```python
# 执行 date --utc 指令
import subprocess
subprocess.run(["date", "--utc"])

# 捕获外部命令输出 text=true是将输出的二进制转换成字符串
p = subprocess.run(['date'], capture_output=True, text=True)
print(p.stdout)  # 打印命令执行结果
```

```python
print(p.returncode)  # 打印命令有没有执行成功
# 检查指令是否执行成功
if p.returncode != 0:
  print(p.stderr)
print(q.stderr)  # 打印错误信息

# 给subprocess传入数据
message = "I love COMP(2041|9044)\n"
p = subprocess.run(["tr", "a-z", "A-Z"], input=message, capture_output=True, text=True)
print(p.stdout)

# 整个字符串给shell解释执行
subprocess.run("sort *.csv | cut -d, -f1,7 >output.txt", shell=True)
```

4 python 正则表达式

4.1 四个基本函数

```python
import re
re.search(r"正则表达式", data)

re.fullmatch(r"正则表达", data)  # 整个字符串完全匹配(常用于判断是不是纯数字)

re.findall(r"正则表达", data)  # 输出是列表 -> 所有匹配的内容

re.sub(r"正则表达式-被替换内容", "替换内容", data)  # 替换
re.sub(r"正则表达式-被替换内容", "\2 \1", data) # \1代表第一个分组(匹配到的第一个内容) \2代表第二个分组

re.split(r"正则表达式-切分原则", data)

# search 案例
s = "abc123xyz"
m = re.search(r"\d+", s)  # 只要里面有就行
print(m)
print(m.group())  # 打印匹配内容

# sub 案例
s = "abc123xyz456"
new_s = re.sub(r"\d+", "#", s)
print(new_s)  # 数字字符串替换成# -> abc#xyz#

```

```
"""
注: re.search(r"^\d+$", data) == re.search(r"\d+", data)
"""
```

4.2 正则化语法

```
匹配普通字符
r"abc"

匹配任意字符
r"a.c"

匹配数字([0-9])
r"\d" -> r"\D"(匹配非数字)

匹配字母、数字、下划线[a-zA-Z0-9_]
r"\w" -> r"\W"

匹配空白字符([[:space:]])
r"\s" -> r"\S"

\b 单词边界
r"\bcat\b" 只会匹配到cat 不会匹配到catch

转移字符: 匹配单个的点
r"\."


前面内容出现0或多次 *
前面内容出现1或多次 +
前面内容出现0或1次 ?   r"-?\d+" 用于判断整数格式（可带负号）

匹配字符集合 跟grep -E一样
[abc] [0-9]
取反
[^...]

或者
r"warning|error"

分组
r"(ab)+"  # ab重复1或多次

匹配开头和结尾
```

```
r"^...$"
```

4.3 正则化案例

```
import re
answer = input()

# flags=re.I 忽略大小写
if re.match(r"yes|ok|affirmative", answer, flags=re.I):
    print("匹配到了")

# 输出匹配内容
m = re.search(r"[aiou].*[aeiou]", "pillow")

m = re.search(r"", data)
print(m.group(0))  # 输出所有匹配的内容
print(m.group(1))  # 输出第一个匹配的内容
print(m.group(2))  # 输出第二个匹配的内容
print(m.span()) # 返回匹配区间

# 使用group的案例 超级重要！！
import re

s = "zxx 185 150 man"
m = re.search(r"\S+\s(\d+)\s+(\d+)\s+man", s)

print(m.group(1))  # 185 输出第一个()中的匹配内容
print(m.group(2))  # 150 输出第二个()中的匹配内容
```

shell 指令复现

1 echo 指令

```
import sys

for i in (1, len(sys.argv)):
  if i > 1:
```

```
    print(" ", end="")
  print(sys.arg[i], end="")
print() # 最后一定要后换行符
```

2 wc -l 指令

```
import sys

lines=sys.stdin.readlines()
lines_count=len(lines)
print(f"行数: {lines_count}")
```

3 cp 指令

```
import sys

if len(sys.argv) != 3:
  print("error")
  sys.exit(1)

in_file = open(sys.argv[1],"r")
out_file = open(sys.argv[2],"w")

# 从文件中一行一行读取
for line in in_file:
  # 将in_file的一行 打印到out_file中
  print(line, end='', file=outfile)

in_file.close()
out_file.close()
```